

Milestone 1.....	1
Milestone 2.....	11
STEP BY STEP GUIDELINE.....	13
KEIL (me) set-up.....	13
STEP BY STEP CONT'D.....	18
AD3 INPUT.....	19
DEBUG PART.....	19
TO RECORD VALUES.....	20
TO SEE VALUES.....	20
Milestone 3.....	20
BONUS Milestone 4.....	21

Milestone 1

Connect the AD3 (or AD2) signal generator (use Wavegen to provide DC values) and complete the table given below. You should convert the binary values into hexadecimal numbers and also provide the percent of maximum digital value. You will need to write your code to copy the converted digital value to a watch variable for your inspection in the Keil debugger. Your TA will be asking about the entire process, i.e., how the ADC value is found, how you captured data, and so on.

For this milestone you may want to use and possibly modify the memory capture method provided in Appendix C. Watching the Lab 3 Pre-Lab video will also be helpful here (see the next milestone for more details on this). You should acquire 5-10 samples for each voltage and then provide the average in the table.

$$Code = \frac{V_{in}}{V_{ref}} (2^{12} - 1)$$

Process:

Hardware Setup

1. Configure waveform generator

- a. Connecting the AD3 to computer with USB
 - b. Select DC on Wavegen
 - c. Test the output voltage of wavegen to ensure output is accurate (0V, 3.3V etc)
2. Configure ADC pin
 - a. Connect microcontroller to computer via USB
 - b. Configure Pin 12, enable PE3 Port E
 - c. Set AMSEL bit for PE3
 - d. Connect wavegen output to ADC input pin (Pin 12 for AIN0)
 - e. Connect wavegen ground to microcontroller ground

Software Setup

```
//Initializing the ADC

// STEP 1: Enable clocks
SYSCTL->RCGCGPIO |= (1 << 4); // Enable Port E clock
SYSCTL->RCGCADC |= (1 << 0); // Enable ADC0 clock
while((SYSCTL->PRADC & (1 << 0)) == 0); // Wait for ADC ready

// STEP 2: Configure PE3 as AIN0
GPIOE->AMSEL |= (1 << 3); // Analog mode
GPIOE->AFSEL |= (1 << 3); // Alternate function
GPIOE->DEN &= ~(1 << 3); // Disable digital

// STEP 3: Configure ADC0 Sequence 3
ADC0->ACTSS &= ~(1 << 3); // Disable seq 3 during config
ADC0->SSMUX3 = 0; // Channel 0 (AIN0)
ADC0->SSCTL3 = 0x06; // END0 + IE0
ADC0->SAC = 4; // 64x hardware averaging
ADC0->ACTSS |= (1 << 3); // Enable seq 3

// STEP 4: Warm-up delay
for(volatile int i = 0; i < 10000; i++);

//Reading result

uint16_t Read_ADC_Single(void) {
```

```

    ADC0->PSSI |= (1 << 3);           // Start conversion
    while((ADC0->RIS & (1 << 3)) == 0); // Wait for completion
    uint16_t result = ADC0->SSFIFO3;   // Read result
    ADC0->ISC = (1 << 3);             // Clear interrupt
    return result;
}

//Averaging samples

uint16_t Read_ADC_Average(void) {
    uint32_t sum = 0;
    uint16_t samples[10];

    for(int i = 0; i < 10; i++) {
        samples[i] = Read_ADC_Single();
        sum += samples[i];
        for(int j = 0; j < 1000; j++); // Small delay
    }

    return (uint16_t)(sum / 10);
}

//Setting up Watch Variables

volatile uint16_t adc_average = 0;
volatile uint32_t adc_hex = 0;
volatile float adc_percent = 0.0;

void Process_Measurement(void) {
    adc_average = Read_ADC_Average();
    adc_hex = adc_average; // Same value, different display
    adc_percent = ((float)adc_average / 4095.0) * 100.0;
}

```

Measurement

- Added variables to watch window
- Recorded decimal, hex, and percentage values

```
// 2DX4StudioW20E1_ADCDemo.c
// This program illustrates the use of the ADC in the C language.
// Note the library headers associated are PLL.h and SysTick.h,
// which define functions and variables used in PLL.c and SysTick.c.
// This program uses code directly from your course textbook.
//
// Milestone 5.1: Data Acquisition System with Multiple ADC Channels
//
// Written by Tom Doyle
// January 18, 2020
// Last Update: Modified for Milestone 5.1

#include <stdint.h>
#include "tm4c1294ncpdt.h"
#include "PLL.h"
#include "SysTick.h"

// Global variables for ADC values
volatile uint32_t ADC_value_ch0; // Channel 0 (PE3)
volatile uint32_t ADC_value_ch1; // Channel 1 (PE2)
volatile uint32_t ADC_value_ch9; // Channel 9 (PE4)

// Arrays to store data for analysis
uint32_t ch0_data[100];
uint32_t ch1_data[100];
uint32_t ch9_data[100];
float ch0_voltage[100];
float ch1_voltage[100];
float ch9_voltage[100];

// -ADC0_InSeq3_SingleChannel-
// Busy-wait ADC conversion for a specific channel
// Input: channel number (0-11)
// Output: 12-bit ADC result
uint32_t ADC0_InSeq3_SingleChannel(uint8_t channel) {
```

```

uint32_t result;

// Configure channel
ADC0_SSMUX3_R = channel & 0x0F;

// Initiate conversion
ADC0_PSSI_R = 0x0008;

// Wait for conversion to complete
while((ADC0_RIS_R & 0x08) == 0) {}

// Read result
result = ADC0_SSFIFO3_R & 0xFFF;

// Acknowledge completion
ADC0_ISC_R = 0x0008;

return result;
}

// Convert ADC value to voltage (0-3.3V)
float ADC_To_Voltage(uint32_t adc_value) {
    return (adc_value * 3.3) / 4095.0;
}

// Initialize ADC for multiple channels
void ADC_MultiChannel_Init(void) {
    // 1. Activate clock for Port E
    SYSCTL_RCGCGPIO_R |= 0b00010000; // Port E
    while ((SYSCTL_PRGPIO_R & 0b00010000) == 0) {}

    // Configure PE2, PE3, PE4 as analog inputs
    // PE2 = AIN1, PE3 = AIN0, PE4 = AIN9
    GPIO_PORTE_DIR_R &= ~0x1C;    // PE2, PE3, PE4 as inputs (bits 2,3,4)
    GPIO_PORTE_AFSEL_R |= 0x1C;    // Enable alternate function
    GPIO_PORTE_DEN_R &= ~0x1C;    // Disable digital I/O
    GPIO_PORTE_AMSEL_R |= 0x1C;    // Enable analog function

    // 2. Activate ADC0
    SYSCTL_RCGCADC_R |= 0x01;

```

```

while((SYSCTL_PRADC_R & 0x01) == 0) {} // Wait for ADC to be ready

// 3. Configure ADC
ADC0_PC_R = 0x01;           // 125K samples/sec
ADC0_SS PRI_R = 0x0123;     // Sequencer 3 is highest priority

// 4. Disable sequencer 3 while configuring
ADC0_ACTSS_R &= ~0x0008;

// 5. Software trigger
ADC0_EMUX_R &= ~0xF000;

// 6. Configure for single channel (we'll change this per sample)
ADC0_SSMUX3_R = 0;         // Default to channel 0
ADC0_SSCTL3_R = 0x0006;    // Single sample, IE0, END0

// 7. Disable interrupts (using polling)
ADC0_IM_R &= ~0x0008;

// 8. Enable sequencer 3
ADC0_ACTSS_R |= 0x0008;
}

// Initialize Port N LED
void PortN_Init(void) {
    SYSCTL_RCGCGPIO_R |= SYSCTL_RCGCGPIO_R12; // Activate Port N
    while((SYSCTL_PRGPIO_R & SYSCTL_PRGPIO_R12) == 0) {}

    GPIO_PORTN_DIR_R |= 0x03;    // PN0 and PN1 as outputs (two LEDs)
    GPIO_PORTN_AFSEL_R &= ~0x03; // Disable alt function
    GPIO_PORTN_DEN_R |= 0x03;    // Enable digital I/O
    GPIO_PORTN_AMSEL_R &= ~0x03; // Disable analog

    // Blink LEDs to show initialization
    GPIO_PORTN_DATA_R |= 0x01;
    SysTick_Wait10ms(10);
    GPIO_PORTN_DATA_R &= ~0x01;
    GPIO_PORTN_DATA_R |= 0x02;
    SysTick_Wait10ms(10);
    GPIO_PORTN_DATA_R &= ~0x02;
}

```

```

}

// Calculate statistics from collected data
void Calculate_Statistics(uint32_t data[], float voltage[], uint32_t count,
                        uint32_t *min_adc, uint32_t *max_adc,
                        float *min_volt, float *max_volt, float *avg_volt) {
    if (count == 0) return;

    *min_adc = data[0];
    *max_adc = data[0];
    *min_volt = voltage[0];
    *max_volt = voltage[0];
    float sum = 0;

    for (uint32_t i = 0; i < count; i++) {
        if (data[i] < *min_adc) *min_adc = data[i];
        if (data[i] > *max_adc) *max_adc = data[i];
        if (voltage[i] < *min_volt) *min_volt = voltage[i];
        if (voltage[i] > *max_volt) *max_volt = voltage[i];
        sum += voltage[i];
    }

    *avg_volt = sum / count;
}

// Display results using LEDs (visual feedback)
void Display_Status(float voltage1, float voltage2, float voltage3) {
    // Simple visualization: Turn on LED1 if any voltage > 2.5V
    // Turn on LED2 if all voltages are stable (< 0.1V difference)

    float avg = (voltage1 + voltage2 + voltage3) / 3.0;
    float max_diff = 0;

    // Find maximum difference
    float diff;
    diff = (voltage1 > avg) ? voltage1 - avg : avg - voltage1;
    if (diff > max_diff) max_diff = diff;
    diff = (voltage2 > avg) ? voltage2 - avg : avg - voltage2;
    if (diff > max_diff) max_diff = diff;
    diff = (voltage3 > avg) ? voltage3 - avg : avg - voltage3;

```

```

if (diff > max_diff) max_diff = diff;

// Control LEDs based on conditions
if (voltage1 > 2.5 || voltage2 > 2.5 || voltage3 > 2.5) {
    GPIO_PORTN_DATA_R |= 0x01; // Turn on LED1
} else {
    GPIO_PORTN_DATA_R &= ~0x01; // Turn off LED1
}

if (max_diff < 0.1) {
    GPIO_PORTN_DATA_R |= 0x02; // Turn on LED2 (stable)
} else {
    GPIO_PORTN_DATA_R &= ~0x02; // Turn off LED2
}
}

int main(void) {
    uint32_t count = 0;
    uint32_t sample_delay = 10; // 100ms between samples

    // Statistics variables
    uint32_t min_adc_ch0, max_adc_ch0, min_adc_ch1, max_adc_ch1, min_adc_ch9,
max_adc_ch9;
    float min_volt_ch0, max_volt_ch0, avg_volt_ch0;
    float min_volt_ch1, max_volt_ch1, avg_volt_ch1;
    float min_volt_ch9, max_volt_ch9, avg_volt_ch9;

    // Initialize system
    PLL_Init();           // 120MHz system clock
    SysTick_Init();       // SysTick for delays
    ADC_MultiChannel_Init(); // ADC for multiple channels
    PortN_Init();         // LEDs for visual feedback

    // Main data acquisition loop
    while (count < 100) {
        // Toggle LED to show sampling activity
        GPIO_PORTN_DATA_R ^= 0x01;

        // Sample Channel 0 (AIN0 on PE3)
        ADC_value_ch0 = ADC0_InSeq3_SingleChannel(0);
    }
}

```



```

ch0_data[count] = ADC_value_ch0;
ch0_voltage[count] = ADC_To_Voltage(ADC_value_ch0);
SysTick_Wait10ms(1); // Small delay between channel samples

// Sample Channel 1 (AIN1 on PE2)
ADC_value_ch1 = ADC0_InSeq3_SingleChannel(1);
ch1_data[count] = ADC_value_ch1;
ch1_voltage[count] = ADC_To_Voltage(ADC_value_ch1);
SysTick_Wait10ms(1);

// Sample Channel 9 (AIN9 on PE4)
ADC_value_ch9 = ADC0_InSeq3_SingleChannel(9);
ch9_data[count] = ADC_value_ch9;
ch9_voltage[count] = ADC_To_Voltage(ADC_value_ch9);

// Provide visual status based on readings
Display_Status(ch0_voltage[count], ch1_voltage[count], ch9_voltage[count]);

// Wait before next sample set
SysTick_Wait10ms(sample_delay);

count++;
}

// After collecting 100 samples, calculate statistics
Calculate_Statistics(ch0_data, ch0_voltage, 100,
    &min_adc_ch0, &max_adc_ch0,
    &min_volt_ch0, &max_volt_ch0, &avg_volt_ch0);

Calculate_Statistics(ch1_data, ch1_voltage, 100,
    &min_adc_ch1, &max_adc_ch1,
    &min_volt_ch1, &max_volt_ch1, &avg_volt_ch1);

Calculate_Statistics(ch9_data, ch9_voltage, 100,
    &min_adc_ch9, &max_adc_ch9,
    &min_volt_ch9, &max_volt_ch9, &avg_volt_ch9);

// Visual indication that data collection is complete
GPIO_PORTN_DATA_R |= 0x03; // Turn on both LEDs
SysTick_Wait10ms(100); // Wait 1 second

```

```
GPIO_PORTN_DATA_R &= ~0x03; // Turn off both LEDs
```

```
// Process complete - enter low-power mode or continuous monitoring
```

```
while (1) {
```

```
    // Continuous monitoring mode
```

```
    ADC_value_ch0 = ADC0_InSeq3_SingleChannel(0);
```

```
    ADC_value_ch1 = ADC0_InSeq3_SingleChannel(1);
```

```
    ADC_value_ch9 = ADC0_InSeq3_SingleChannel(9);
```

```
    // Update display based on current readings
```

```
    Display_Status(ADC_To_Voltage(ADC_value_ch0),
```

```
        ADC_To_Voltage(ADC_value_ch1),
```

```
        ADC_To_Voltage(ADC_value_ch9));
```

```
    SysTick_Wait10ms(50); // Check every 500ms
```

```
}
```

```
}
```

```
//1. Potentiometer ? PE3 (Channel 0) // Variable voltage
```

```
//2. 3.3V ? PE2 (Channel 1) // Always 3.3V (max reading)
```

```
//3. GND ? PE4 (Channel 9) // Always 0V (min reading)
```

Analog Voltage (V _H) DC	Decimal Value	X-bit ADC (hex)	Percent of Max Value (%)
0.00	3	0x0003	0.07
0.33	408	0x198	9.96
0.66	816	0x330	19.93
1.00	1237	0x4D5	30.21
1.33	1646	0x66E	40.20
1.66	2056	0x808	50.21
2.00	2477	0x9AD	60.49
2.33	2887	0xB47	70.50
2.66	3296	0xCE0	80.49

3.00	3719	0xE87	90.82
3.30	4095	0xFFF	100.00

Possible TA questions

1. How did you configure the ADC
 - a. We enabled clocks, configured pin PE3 with AMSEL and AFSEL
2. Why average 5-10 samples
 - a. Reduces random noise
3. How did you read the ADC value
 - a. Software trigger starts conversion
 - b. Wait for RIS flag
 - c. Read from SSFIF03
 - d. Clear interrupt with ISC register

Milestone 2

Before proceeding, check the following:

1. Ensure the sinusoid signal is within the voltage range of the ADC (V_{RHN} , R_{RHP})
2. Ensure the signal generator output and the microcontroller power supply are connected to the same reference

Add the student numbers of the lab group together and observe the least significant digit (LSD) of the sum. Refer to the table below for the assigned sinusoid frequency.

400582489

400559483

Sum: 801141972

LSD = 2

Therefore, use 240 Hz sinusoid freq

Microcontroller uses 120 MHz

To calculate delay, simply do:

$$120\text{MHz} / (240 \text{ Hz} * 10)$$

This is 50,000

Sum Least Significant Digit	0-3V Sinusoid Frequency (Hz)
0, 3, 6, 9	80
1, 4, 7	160
2, 5, 8	240

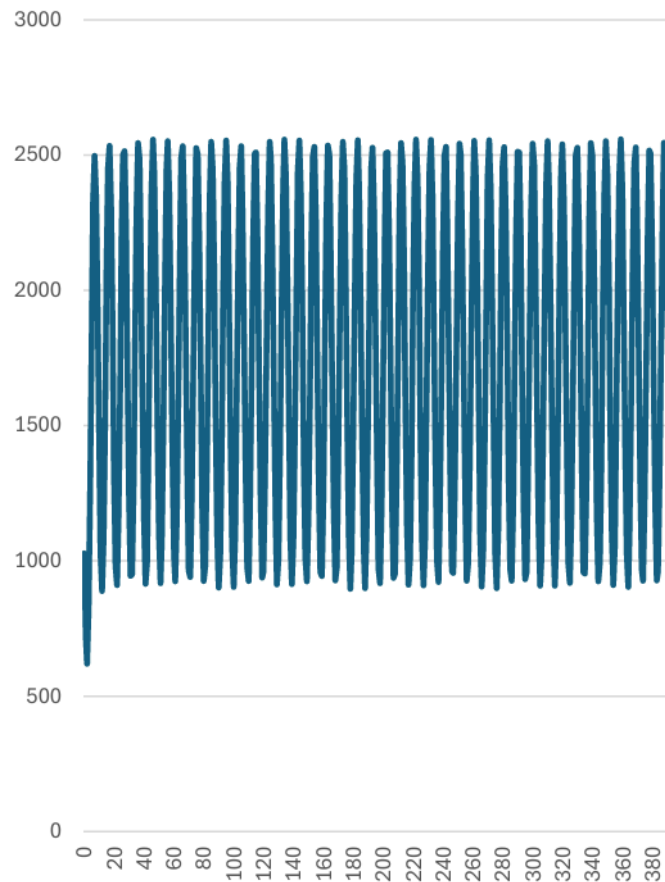
Configure the signal generator (AD3 or AD2 Wavegen) to output a sinusoid of 3Vpp (0-3V DC) at the assigned frequency. Configure the ADC to sample the sinusoid without aliasing (reproduce frequency content). Clearly state your sampling rate and explain how this was achieved in your code to the TA.

In Studio we were able to use the functional debugging approach by setting up an IDE Watch where a global array was defined and it could be inspected during execution and post-execution.

Our data of interest was small enough that we could transcribe by hand the values into an excel document for plotting. In this experiment, given our sampling rate, a meaningful duration of time will mean hundreds of data points which will be impossible to transcribe during your lab time. Instead we will use Keil's Command Line tools to export our data to a file for analysis – see the Lab 3 - Pre-Lab video (Avenue > Content > Lab Resources > Lab 3) for a demonstration of using the Keil Command Line for exporting data from our microcontroller's memory. The MemoryValues script can also be found in the same Avenue folder. You can also see Appendix C of the Lab Manual for assistance with this process including information on Keil debug commands: LOG, EVAL, D, FUNC, and creating .ini files.

Plot 500ms of captured waveform data and have it evaluated by the TA.

STEP BY STEP GUIDELINE



- 1.) Connect AD3 and microcontroller to computer
- 2.) Directly connect AD3 ground to ground on microcontroller (no need for extra wires)
- 3.) Directly connect AD3 W1 (waveform) to PE3 or PE4 (no need for extra wires)
 - a.) Whether you choose PE3 or PE4 is up to you
 - b.) I believe the professor's code used PE4
- 4.) Now that you have finished with hardware set-up, proceed with the following:

KEIL (me) set-up

1. Check to see if you have all the correct files selected
 - a. PLL.c
 - b. PLL.h
 - c. SysTick.c
 - d. SysTick.h

- e. (your own C file for code implementation). Do not choose C++ or else you WILL encounter errors
 - f. The files from 'a' to 'd' can be found under Studio 3 (do not quote me on that)
- 2. Do not be stupid. Remember to select Startup under 'Devices' (no need to add Startup.s *manually*)
- 3. Firstly, go to SysTick.c
 - a. You will notice that at the very bottom starting at line 67 (yes, that was intentional), you will have a void function called SysTick_Wait10ms
 - b. WE DON'T WANT 10ms
 - c. It's too long of a wait
 - d. Currently, this is what I have near the end of this SysTick.c file (I don't remember if it has been modified)

```
// Time delay using busy wait.
// The delay parameter is in units of the core clock. (units of 8.333 nsec for 120 MHz clock)
void SysTick_Wait(uint32_t delay){
    volatile uint32_t elapsedTime;
    uint32_t startTime = NVIC_ST_CURRENT_R;
    do{
        elapsedTime = (startTime-NVIC_ST_CURRENT_R)&0x00FFFFFF;
    }
    while(elapsedTime <= delay);
}
// Time delay using busy wait.
// This assumes 120 MHz system clock.
void SysTick_Wait10ms(uint32_t delay){
    uint32_t i;
    for(i=0; i<delay; i++){
        SysTick_Wait(1200000); // wait 10ms (assumes 120 MHz clock)
    }
}
```

- 4. We will implement SysTick_Wait (line 57) into our code
- 5. Now, take a look at the code below (essentially the Studio Code modified)
 - a. Scroll all the way down to the very end of the code for more details

```
// 2DX4StudioW20E1_ADCDemo
// This program illustrates the use of the ADC in the C language.
// Note the library headers associated are PLL.h and SysTick.h,
```

```

// which define functions and variables used in PLL.c and SysTick.c.
// This program uses code directly from your course textbook.
//
// This example will be extended for in W21E0 and W21E1.
//
// Written by Tom Doyle
// January 18, 2020
// Last Update: January 21, 2020

#include <stdint.h>
#include "tm4c1294ncpdt.h"
#include "PLL.h"
#include "SysTick.h"

volatile uint32_t ADCvalue;

// -ADC0_InSeq3-
// Busy-wait analog to digital conversion. 0 to 3.3V maps to 0 to 4095
// Input: none
// Output: 12-bit result of ADC conversion
uint32_t ADC0_InSeq3(void){
    uint32_t result;

    ADC0_PSSI_R = 0x0008;
                                // 1) initiate SS3
    while((ADC0_RIS_R&0x08)==0){}
        // 2) wait for conversion done
    result = ADC0_SSFIFO3_R&0xFFF;
        // 3) read 12-bit result
    ADC0_ISC_R = 0x0008;
                                // 4) acknowledge completion

    return result;
}

void ADC_Init(void){
    //config the ADC from Valvano textbook
    SYSTL_RCGCGPIO_R |= 0b00010000;
    // 1. activate clock for port E

```

```

        while ((SYSCTL_PRGPIO_R & 0b00010000) == 0) {} // wait for
clock/port to be ready
        GPIO_PORTE_DIR_R &= ~0x10;
            // 2) make PE4 input
        GPIO_PORTE_AFSEL_R |= 0x10;
            // 3) enable alternate function on PE4
        GPIO_PORTE_DEN_R &= ~0x10;
            // 4) disable digital I/O on PE4
        GPIO_PORTE_AMSEL_R |= 0x10;
            // 5) enable analog function on PE4
        SYSCTL_RCGCADC_R |= 0x01;
            // 6) activate ADC0
        ADC0_PC_R = 0x01;
            // 7) maximum speed is 125K samples/sec
        ADC0_SSPRI_R = 0x0123;
            // 8) Sequencer 3 is highest priority
        ADC0_ACTSS_R &= ~0x0008;
            // 9) disable sample sequencer 3
        ADC0_EMUX_R &= ~0xF000;
            // 10) seq3 is software trigger
        ADC0_SSMUX3_R = 9;
            // 11) set channel Ain9 (PE4)
        ADC0_SSCTL3_R = 0x0006;
            // 12) no TS0 D0, yes IE0 END0
        ADC0_IM_R &= ~0x0008;
            // 13) disable SS3 interrupts
        ADC0_ACTSS_R |= 0x0008;
            // 14) enable sample sequencer 3

//=====
=====

    return;
}

void PortN_Init(void){
    //Use PortN onboard LED
    SYSCTL_RCGCGPIO_R |= SYSCTL_RCGCGPIO_R12; //
    activate clock for Port N

```



```
while((SYSCTL_PRGPIO_R&SYSCTL_PRGPIO_R12) == 0){}; // allow time for clock
to stabilize
```

```
GPIO_PORTN_DIR_R |= 0x01;
// make PN0 out (PN0 built-in LED1)
GPIO_PORTN_AFSEL_R &= ~0x01;
// disable alt funct on PN0
GPIO_PORTN_DEN_R |= 0x01;
// enable digital I/O on PN0
```

```
//
```

```
configure PN1 as GPIO
```

```
//GPIO_PORTN_PCTL_R = (GPIO_PORTN_PCTL_R&0xFFFFF0F)+0x00000000;
GPIO_PORTN_AMSEL_R &= ~0x01;
// disable analog functionality on PN0
```

```
GPIO_PORTN_DATA_R ^= 0b00000001;
//hello world!
SysTick_Wait10ms(10);
//.1s delay
GPIO_PORTN_DATA_R ^= 0b00000001;
return;
}
```

```
uint32_t func_debug[1200];
// array used to store realtime data
```

```
int main(void){
```

```
uint32_t delay = 50000,
// this is a multiplier used for SysTick = half of delay
count = 0;
// my loop control variable initialized
to 0
```

```
PLL_Init();
// Default Set System Clock to
120MHz
SysTick_Init();
// Initialize SysTick configuration
```

```

ADC_Init();
// Initialize ADC as per Vavano text

PortN_Init();
// Initialize Port N GPIO

while(count < 1200){
    // run program for fixed number of steps/samples
    GPIO_PORTN_DATA_R ^= 0b000000001;
    // toggle LED for visualization of process
    SysTick_Wait(delay);
// halfdelay*10ms
    func_debug[count++] = ADC0_InSeq3();
Acquire sample
}

}

```

STEP BY STEP CONT'D

1. Now that you have (hopefully) read through the code, take a good look at line 77
 - a. You do realize that uint32_t delay has been set to 50000?
 - b. Yes, please ask why
 - c. Why?

Microcontroller uses 120 MHz

To calculate delay, simply do:

$120\text{MHz} / (240 \text{ Hz} * 10)$

This is 50,000

Where did 240 Hz come from? It came from Nejat's hat.

No, stupid. It came from the fact that we were ASSIGNED to use this frequency.

If your student (lol almost wrote stupid) numbers added up together uses a different frequency (refer to chart at the very beginning of M2), use YOUR assigned frequency.

2. Notice line 75

```
uint32_t func_debug[1200];  
    // array used to store realtime data
```

This allocates 1200 ‘space’ for your while loop in line 86 to run however many times

Think about why 1200

After you are done thinking, you should know what number to modify in the while loop of line 86. *Hint: it's not the number 67*

In addition to that, modify line 88 to just SysTick_Wait

Don't use the one with 10 ms delay

Now compile and load

AD3 INPUT

1. Simply open wavegen and choose the frequency you were assigned
2. Modify amplitude = 1.5V and offset = 1.5V
3. Turn it on (run it)

Now go back to KEIL

DEBUG PART

1. Open debugger
2. Click ‘debug’
3. Select (i think it was the 2nd last one) whatever the prelab video showed you
4. You will be transported into another dimension
5. Add this: (this is a picture btw, please don't be a lazy bum - go to D2L and lab materials to find for yourself)

Memory Value Capture Function

```
FUNC void memoryvalues(void) {  
  
    int i;  
  
    //formatted decimal display of the array contents  
    exec("log > 2DX3Data.txt");  
    for(i = 0; i < 100; i++)  
    {  
        printf("array[%04u] = %04u\n", i, func_debug[i]);  
    }  
    exec("log off");  
  
    //array contents suitable for csv file  
    exec("log > 2DX3Data.csv");  
  
    for(i = 0; i < 100; i++)  
    {  
        printf("%u,%u\n", i, func_debug[i]);  
    }  
    exec("log off");  
}
```

6. If you need help with this, ask a TA
7. I have short term memory loss and it is difficult for me to compile all of this already.
Especially after a stupid midterm.
8. Anyways, moving on

Change the values in the 'for' loop in this memory capture function to how many data points you want

In our case, it was 1200

Save, compile

TO RECORD VALUES

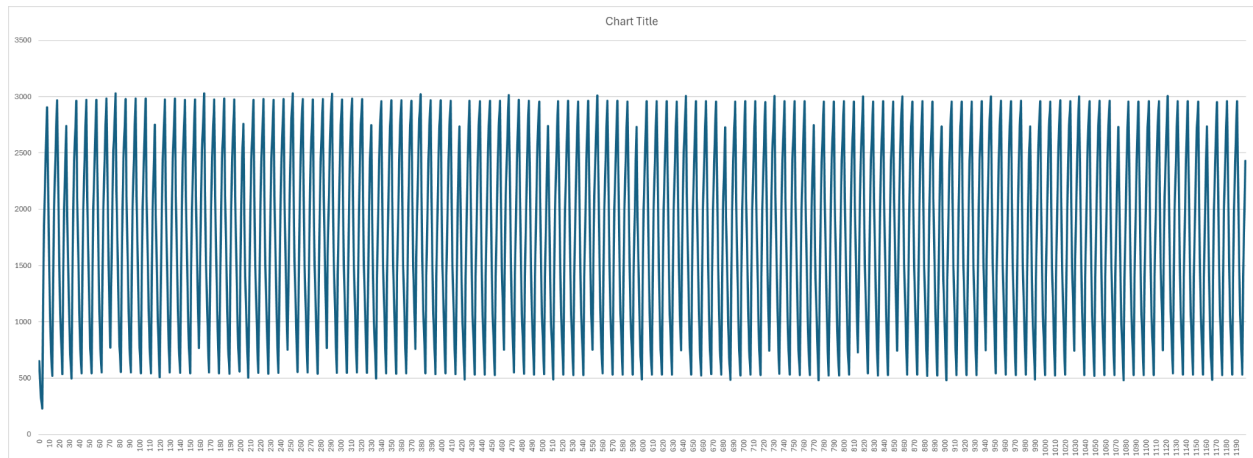
- Remember to hit 'run' on Wavegen if you haven't already
- Remember to hit 'run' on Keil (2nd button from somewhere near the top left)
- Enter memoryvalues() into the small, long rectangular box underneath (i think it was the command box?)
- You should be seeing that near bottom left

TO SEE VALUES

- Find it in your files
- You should know how to make a graph out of the data points. It is very simple.

Milestone 3

Configure the signal generator (AD3 or AD2 Wavegen) to output a square wave of 3Vpp (0-3V DC) at the assigned frequency. Configure the ADC to sample at the same frequency you used for the previous milestone. Explain the result to the TA. Plot 500ms of captured waveform data and have it evaluated by the TA.



BONUS Milestone 4

Write a program to calculate the frequency of a sinusoidal waveform which is assumed to be non-aliased up to 100 Hz. Your TA will ask you to modify the signal generator frequency to a value of 100 Hz or less and your program must automatically measure and update the frequency count. Use the functional debugging approach where we employed a global variable to set up an IDE Watch in Studio that was inspected during execution and post-execution. Display the waveform frequency that your program computes in such a variable.

```
// =====
// Milestone 5.4.4: UART Data Transmission for MSP-EXP432E401Y
// Reads ADC channels and transmits data to PC via UART
// Compatible with MSP432E401Y microcontroller
// =====

#include <stdint.h>
#include <stdio.h>
#include <string.h>
#include <ti/devices/msp432e4/driverlib/driverlib.h>

// ===== GLOBAL DEBUG VARIABLES =====
volatile uint32_t g_debug_adc_ch0 = 0;
volatile uint32_t g_debug_adc_ch1 = 0;
volatile uint32_t g_debug_adc_ch9 = 0;
volatile float g_debug_voltage_ch0 = 0.0;
volatile float g_debug_voltage_ch1 = 0.0;
volatile float g_debug_voltage_ch9 = 0.0;
volatile uint32_t g_debug_uart_tx_count = 0;
```

```
// ===== BOARD-SPECIFIC CONFIGURATION =====
// =====
// MSP-EXP432E401Y Pin Mapping:
// ADC0: PE3 (Analog channel 0) - Use for potentiometer 1
// ADC1: PE2 (Analog channel 1) - Use for potentiometer 2
// ADC9: PE4 (Analog channel 9) - Use for potentiometer 3
// UART0: PA0 (RX), PA1 (TX) - For serial communication
// LED1: PN0 (Green LED)
// LED2: PN1 (Blue LED)

#define ADC_SAMPLE_RATE_HZ 1000
#define UART_BAUD_RATE 115200
#define SYSTEM_CLOCK 120000000 // 120 MHz

// ===== FUNCTION PROTOTYPES =====
void Init_SystemClock(void);
void Init_ADC_MultiChannel(void);
void Init_UART0(void);
void Init_GPIO_LEDs(void);
void UART_PrintString(const char *str);
void UART_PrintData(void);
float ADC_to_Voltage(uint32_t adc_value);

// ===== SYSTEM INITIALIZATION =====
void Init_SystemClock(void) {
    // Set the clock to 120 MHz
    MAP_SysCtlClockFreqSet((SYSCTL_XTAL_25MHZ | SYSCTL_OSC_MAIN |
        SYSCTL_USE_PLL | SYSCTL_CFG_VCO_480), 120000000);
}

// ===== ADC INITIALIZATION =====
void Init_ADC_MultiChannel(void) {
    // Enable ADC0 peripheral
    MAP_SysCtlPeripheralEnable(SYSCTL_PERIPH_ADC0);
    while(!MAP_SysCtlPeripheralReady(SYSCTL_PERIPH_ADC0)) {}

    // Enable GPIO Port E for ADC inputs
    MAP_SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOE);
    while(!MAP_SysCtlPeripheralReady(SYSCTL_PERIPH_GPIOE)) {}

    // Configure PE2, PE3, PE4 as analog inputs
    // PE2 = AIN1, PE3 = AIN0, PE4 = AIN9

```

```

    MAP_GPIODirModeSet(GPIO_PORTE_BASE, GPIO_PIN_2 | GPIO_PIN_3 | GPIO_PIN_4,
GPIO_DIR_MODE_HW);
    MAP_GPIOPadConfigSet(GPIO_PORTE_BASE, GPIO_PIN_2 | GPIO_PIN_3 |
GPIO_PIN_4,
        GPIO_STRENGTH_2MA, GPIO_PIN_TYPE_ANALOG);

// Disable ADC0 sequencer 3 during configuration
MAP_ADCSequenceDisable(ADC0_BASE, 3);

// Configure sequencer 3 for single conversion
MAP_ADCSequenceConfigure(ADC0_BASE, 3, ADC_TRIGGER_PROCESSOR, 0);

// Configure step 0 for channel 0 (PE3)
MAP_ADCSequenceStepConfigure(ADC0_BASE, 3, 0, ADC_CTL_CH0 | ADC_CTL_IE |
ADC_CTL_END);

// Enable ADC0 sequencer 3
MAP_ADCSequenceEnable(ADC0_BASE, 3);

// Clear any pending interrupts
MAP_ADCIntClear(ADC0_BASE, 3);
}

// ===== UART INITIALIZATION =====
void Init_UART0(void) {
    // Enable UART0 and GPIO Port A peripherals
    MAP_SysCtlPeripheralEnable(SYSCTL_PERIPH_UART0);
    MAP_SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOA);

    // Wait for peripherals to be ready
    while(!MAP_SysCtlPeripheralReady(SYSCTL_PERIPH_UART0)) {}
    while(!MAP_SysCtlPeripheralReady(SYSCTL_PERIPH_GPIOA)) {}

    // Configure GPIO pins for UART0
    // PA0 = U0RX, PA1 = U0TX
    MAP_GPIOPinConfigure(GPIO_PA0_U0RX);
    MAP_GPIOPinConfigure(GPIO_PA1_U0TX);
    MAP_GPIOPinTypeUART(GPIO_PORTA_BASE, GPIO_PIN_0 | GPIO_PIN_1);

    // Configure UART0
    // 120 MHz system clock, 115200 baud rate
    // BRD = 120,000,000 / (16 * 115200) = 65.104
    MAP_UARTConfigSetExpClk(UART0_BASE, SYSTEM_CLOCK, 115200,

```

```

        (UART_CONFIG_WLEN_8 | UART_CONFIG_STOP_ONE |
UART_CONFIG_PAR_NONE));

    // Enable UART0
    MAP_UARTEnable(UART0_BASE);
}

// ===== GPIO LED INITIALIZATION =====
void Init_GPIO_LEDs(void) {
    // Enable GPIO Port N for LEDs
    MAP_SysCtlPeripheralEnable(SYSCTL_PERIPH_GPION);
    while(!MAP_SysCtlPeripheralReady(SYSCTL_PERIPH_GPION)) {}

    // Configure PN0 and PN1 as outputs for LEDs
    MAP_GPIOPinTypeGPIOOutput(GPIO_PORTN_BASE, GPIO_PIN_0 | GPIO_PIN_1);

    // Turn off LEDs initially
    MAP_GPIOPinWrite(GPIO_PORTN_BASE, GPIO_PIN_0 | GPIO_PIN_1, 0);
}

// ===== ADC CONVERSION FUNCTION =====
float ADC_to_Voltage(uint32_t adc_value) {
    // Convert 12-bit ADC value (0-4095) to voltage (0-3.3V)
    return (adc_value * 3.3f) / 4095.0f;
}

// ===== UART TRANSMISSION FUNCTIONS =====
void UART_PrintChar(char c) {
    // Wait until TX FIFO is not full
    while(MAP_UARTBusy(UART0_BASE)) {}

    // Send the character
    MAP_UARTCharPut(UART0_BASE, c);
}

void UART_PrintString(const char *str) {
    while(*str) {
        UART_PrintChar(*str);
        str++;
    }
}

void UART_PrintData(void) {

```



```

char buffer[128];

// Format and send the data
snprintf(buffer, sizeof(buffer),
    "ADC Data: CH0=%4u (%1.3fV), CH1=%4u (%1.3fV), CH9=%4u (%1.3fV)\r\n",
    g_debug_adc_ch0, g_debug_voltage_ch0,
    g_debug_adc_ch1, g_debug_voltage_ch1,
    g_debug_adc_ch9, g_debug_voltage_ch9);

UART_PrintString(buffer);

// Update transmission counter
g_debug_uart_tx_count++;
}

// ===== ADC READ FUNCTIONS =====
uint32_t Read_ADC_Channel(uint8_t channel) {
    uint32_t adc_value;

    // Disable sequencer during reconfiguration
    MAP_ADCSequenceDisable(ADC0_BASE, 3);

    // Reconfigure sequencer for the desired channel
    MAP_ADCSequenceConfigure(ADC0_BASE, 3, ADC_TRIGGER_PROCESSOR, 0);

    // Configure step for the specific channel

```